

## ?# Documentacion del proyecto PDI SLP Cyber Defense System

Este documento explica como esta armado el programa y que hace cada parte. La idea es que se pueda seguir el flujo sin tener que leer todo el codigo linea por linea.

## 1. Objetivo del sistema

El proyecto simula un sistema de defensa cibernetica para tres centrales:

? CENTRAL\_NORTE  
? CENTRAL\_CENTRO  
? CENTRAL\_SUR

Durante 60 segundos se genera trafico normal, a veces se inyectan ataques, se calculan metricas matematicas y fisicas, se actualiza el estado del sistema y al final se generan reportes.

El archivo principal es:

m00\_pdi\_slp.py

Los reportes finales son:

pdi\_slp\_graficas/  
reportes\_tabla/pdi\_slp\_reporte\_tabla.csv

La carpeta pdi\_slp\_graficas contiene una imagen individual por grafica.

## 2. Archivos del programa

| Archivo                  | Para que sirve                                         |
|--------------------------|--------------------------------------------------------|
| m00_pdi_slp.py           | Ejecuta la simulacion principal.                       |
| m01_servidores.py        | Guarda datos fijos, agentes, IPs y umbrales.           |
| m02_trafico_normal.py    | Genera actividad normal del sistema.                   |
| m03_atacante.py          | Modifica un paquete normal para simular ataques.       |
| m04_logica_matematica.py | Contiene calculos, alertas, estados, cifrado y fisica. |
| m05_monitor.py           | Muestra informacion en consola.                        |
| m06_graficas.py          | Genera las imagenes de las graficas.                   |
| m07_reporte_tabla.py     | Genera el archivo CSV.                                 |

## 3. Librerias usadas

```
| Libreria | Uso |
|---|---|
| random | Elegir datos aleatorios y decidir si ocurre un ataque. |
| time | Pausar la simulacion para simular segundos reales. |
| math | Usar formulas como raiz, seno, coseno, exponencial y angulos. |
| os | Crear la carpeta donde se guardan las graficas. |
| csv | Crear el reporte tabular. |
| matplotlib | Dibujar y guardar las graficas como imagenes. |
```

Los demas imports son archivos del mismo proyecto.

## 4. Estado inicial

En m00\_pdi\_slp.py se crea un diccionario llamado estado\_actual.

```
estado_actual = {
    "estado": "NORMAL",
    "cifrado": "NINGUNO",
    "refrigeracion": False,
    "login_bloqueado": False,
    "ips_bloqueadas": [],
    "historial": []
}
```

Ese diccionario guarda como se encuentra el sistema durante la simulacion.

Tambien se crea estado\_calculo, que guarda valores anteriores para calcular derivadas:

```
paquetes_anteriores
temperatura_anterior
cpu_anterior
primera_derivada_sigmoide_anterior
```

## 5. Ciclo principal

El programa trabaja con un for:

```
for segundo in range(1, 61):
```

Eso hace que la simulacion dure 60 ciclos. Cada ciclo representa un segundo.

En cada segundo pasa esto:

1. Se genera trafico normal.

2. Se calcula si ocurre un ataque.
3. Si ocurre, se modifica el paquete.
4. Se calculan derivadas, flujo e índice oscilatorio.
5. Se calcula CPU y temperatura.
6. Se evalúa el estado.
7. Se activa cifrado, bloqueo o refrigeración si hace falta.
8. Se guardan los datos en el historial.
9. Se imprime información en consola.
10. Al final se generan gráficas y CSV.

## 6. Trafico normal

m02\_trafico\_normal.py genera un paquete normal. El paquete es un diccionario con datos como:

```
segundo
central
usuario
ip
expediente
paquetes
intentos_login
cambios_archivos
salida_datos
cpu_base
incidente
tipo_incidente
```

Los valores normales están dentro de rangos bajos. Por ejemplo:

```
paquetes: 80 a 420
intentos_login: 0 a 2
cambios_archivos: 0 a 3
salida_datos: 1 a 12 MB
```

## 7. Ataques

m03\_atacante.py puede modificar el paquete normal. Los ataques posibles son:

```
ATAQUE_TRAFICO
FUERZA_BRUTA
ACCESO_NO_AUTORIZADO
COPIA_EXPEDIENTE
MODIFICACION_EVIDENCIA
ELIMINACION_ARCHIVO
EXTRACCION_INFORMACION
```

El ataque escala con el tiempo:

```
factor = segundo / 10
```

Eso significa que un ataque puede ser mas fuerte conforme avanza la simulacion.

El atacante no cambia directamente la temperatura. Lo que hace es subir paquetes, intentos, cambios o salida de datos. Luego el CPU sube por esa actividad y la temperatura sube como consecuencia.

## 8. Probabilidad de ataque

En m00\_pdi\_slp.py la probabilidad cambia por tramos:

```
segundo 1: 0%
segundos 2 a 20: 20%
segundos 21 a 40: 40%
segundos 41 a 60: 60%
```

El primer segundo queda sin ataque para que el sistema inicie en estado normal.

## 9. Calculo

La parte de calculo esta en m04\_logica\_matematica.py.

La tasa compara paquetes actuales contra paquetes anteriores:

```
tasa = paquetes_actuales - paquetes_anteriores
```

La primera derivada discreta mide el cambio entre ciclos:

$$P'(t) = P(t) - P(t-1)$$

La segunda derivada mide si el cambio se acelera:

$$P''(t) = P'(t) - P'(t-1)$$

Luego se aplica una sigmoide como compuerta:

```
sigmoide(x) = 1 / (1 + e^(-x))  
resultado = valor * sigmoide(k * valor / escala)
```

Esta compuerta deja ver mejor las subidas fuertes y suaviza mas las bajadas.

## 10. Algebra

Se usa el indice de flujo digital:

$$F = 0.45P + 0.25L + 0.15C + 0.15D$$

Donde:

P = paquetes  
L = intentos\_login  
C = cambios\_archivos  
D = salida\_datos

El flujo sirve para resumir la intensidad del evento en un solo valor. Esta es la parte algebraica principal del sistema.

## 11. Trigonometria y oscilacion

La grafica 7 usa una onda de envio regular. Ademas, el programa calcula un indice oscilatorio para apoyar la evaluacion del estado.

La onda se genera por intervalos de 5 segundos. En cada intervalo se toma el valor maximo de paquetes, pero se limita a 500 para representar comportamiento regular:

```
amplitud = min(max(paquetes_en_5_segundos), 500)
```

Luego se crea la onda con seno:

```
onda = amplitud * sen(pi * progreso)
```

La linea real de paquetes se compara contra esa onda. Si la linea real se sale demasiado, se interpreta como comportamiento irregular.

El indice oscilatorio usa la diferencia entre la senal periodica y los paquetes reales. Tiene dos niveles:

```
U_OSCILACION_SOSP = 250
U_OSCILACION_CRIT = 1200
```

Si el indice pasa el nivel sospechoso, puede apoyar un estado SOSPECHOSO. Si pasa el nivel critico, puede activar CRITICO y registrar una desviacion oscilatoria critica.

## 12. Fisica: CPU y temperatura

El CPU se calcula a partir de la actividad:

```
cpu = cpu_base + 0.005 * (P + 45L + 28C + 18D)
```

La temperatura se calcula con:

```
T = 28 + 0.58 * CPU
```

Si no hay ataques fuertes, el CPU se mantiene moderado y la temperatura normalmente no llega a refrigeracion activa.

## 13. Fisica: continuidad y Bernoulli

El sistema de refrigeracion usa continuidad:

```
A1 * v1 = A2 * v2
v2 = (A1 * v1) / A2
```

Con los datos del programa:

```
A1 = 0.05 m2
A2 = 0.02 m2
v1 = 2.0 m/s
```

$$v_2 = 5.0 \text{ m/s}$$

Despues usa Bernoulli simplificado:

$$P_2 = P_1 + 0.5 \cdot \rho \cdot v_1^2 - 0.5 \cdot \rho \cdot v_2^2$$

Con los datos:

$$\begin{aligned} P_1 &= 101325 \text{ Pa} \\ \rho &= 1000 \text{ kg/m}^3 \\ v_1 &= 2.0 \text{ m/s} \\ v_2 &= 5.0 \text{ m/s} \\ P_2 &= 90825 \text{ Pa} \end{aligned}$$

## 14. Enfriamiento

Hay dos tipos de enfriamiento.

Enfriamiento pasivo:

si no hay refrigeracion activa,  
cada 5 segundos baja 1 C

Enfriamiento activo:

$$\Delta T = \text{coef} * v_2 * ((P_1 - P_2) / P_1)$$

Con los valores actuales:

$$\begin{aligned} \Delta T &= 4.4 * 5.0 * ((101325 - 90825) / 101325) \\ \Delta T &= 2.28 \text{ C aprox.} \end{aligned}$$

Si el estado es CRITICO, se busca que la temperatura quede por debajo del umbral preventivo:

$$\begin{aligned} \text{objetivo\_termico} &= U\_TEMP\_REFRIG - 1 \\ \text{objetivo\_termico} &= 57 \text{ C} \end{aligned}$$

La refrigeracion ya no queda encendida para siempre. En cada ciclo se vuelve a decidir si hace falta.

## 15. Estados del sistema

Los estados posibles son:

NORMAL  
SOSPECHOSO  
ALERTA  
CRITICO

Se deciden en evaluar\_estado.

| Estado     | Cuando aparece                                                                       |
|------------|--------------------------------------------------------------------------------------|
| NORMAL     | No hay alertas y los valores estan controlados.                                      |
| SOSPECHOSO | No hay alerta fuerte, pero paquetes o login salen un poco de lo normal.              |
| ALERTA     | Hay usuario no autorizado, fuerza bruta, cambios de archivos o salida de datos alta. |
| CRITICO    | Paquetes, flujo o temperatura llegan a nivel critico.                                |

## 16. Alertas y protecciones

Las reglas principales son:

| Condicion                         | Resultado                                            |
|-----------------------------------|------------------------------------------------------|
| Usuario no autorizado             | Alerta y posible bloqueo de IP.                      |
| Paquetes altos + login alto       | Alerta por trafico masivo y fuerza bruta.            |
| Cambios de archivos altos         | Alerta por manipulacion de expediente.               |
| Salida de datos alta              | Alerta y cifrado si no habia cifrado.                |
| Paquetes criticos o flujo critico | Estado CRITICO y cifrado reforzado.                  |
| Temperatura critica               | Estado CRITICO y refrigeracion.                      |
| Login critico                     | Cierre de login remoto.                              |
| Indice oscilatorio critico        | Estado CRITICO y registro de desviacion oscilatoria. |

## 17. Cifrado

El sistema no cifra todo el contenido de un archivo real. En esta simulacion cifra el nombre del registro que se genera para guardar evidencia del ciclo.

Hay tres niveles:

NINGUNO  
CIFRADO  
REFORZADO



## NINGUNO

Se usa cuando el sistema esta normal y no necesita proteccion extra.

Ejemplo de nombre base:

`datos_05.log`

Direccion generada:

`registro_local://central_norte/ci-2025-123456/datos_05.log`

## CIFRADO

Se usa cuando el sistema esta en SOSPECHOSO o ALERTA, o cuando ya se activo cifrado por salida de datos o trafico riesgoso.

En el sistema se muestra como CIFRADO. Por dentro usa cifrado Cesar, moviendo cada letra 3 posiciones en el alfabeto. Los numeros, guiones bajos y puntos se quedan igual.

Ejemplo visible del cifrado:

Antes : `datos_05.log`  
Despues: `gdwrv_05.orj`

Comparacion letra por letra:

`d a t o s _ 0 5 . l o g`  
`g d w r v _ 0 5 . o r j`

La ruta final quedaria asi:

`registro_seguro://central_norte/ci-2025-123456/gdwr_v_05.orj`

## REFORZADO

Se usa cuando el sistema llega a estado CRITICO o cuando ya tenia cifrado reforzado activo.

El cifrado reforzado hace dos pasos:

1. Aplica cifrado Cesar +3
2. Aplica cifrado multiplicativo con clave 5

En el segundo paso, las letras se transforman dentro del alfabeto y los numeros se transforman dentro de las cifras 0 a 9.

Ejemplo visible del cifrado reforzado:

Antes : datos\_05.log  
Despues: epghb\_38.sht

Primer paso, cifrado Cesar +3:

datos\_05.log -> gdwrv\_05.orj

Segundo paso, cifrado multiplicativo con clave 5:

gdwrv\_05.orj -> epghb\_38.sht

En los numeros tambien se nota el cambio:

0 -> 3  
5 -> 8

La ruta final quedaria asi:

registro\_reforzado://central\_norte/ci-2025-123456/epghb\_38.sht

La direccion del registro cambia segun el estado:

NORMAL -> registro\_local://...  
ALERTA o SOSPECHOSO -> registro\_seguro://...  
CRITICO -> registro\_reforzado://...

En resumen, el cifrado sirve para representar que, mientras mas riesgoso es el estado, mas protegida queda la ruta del registro.

## 18. Bloqueo de IP y login

Si el usuario no esta autorizado, su IP se agrega a:

```
estado_actual["ips_bloqueadas"]
```

Si los intentos de login superan el umbral critico, se activa:

```
estado_actual["login_bloqueado"] = True
```

## 19. Historial

Cada ciclo se guarda en:

```
estado_actual["historial"]
```

Ese historial es lo que despues usan las graficas y el CSV.

Campos principales guardados:

```
segundo  
central  
usuario  
ip  
paquetes  
intentos_login  
cambios_archivos  
salida_datos  
cpu  
temperatura  
temperatura_sin_enfriamiento  
refrigeracion_activa  
flujo  
primera_derivada_paquetes  
primera_derivada_sigmoide  
segunda_derivada_sigmoide  
derivada_cpu  
derivada_temperatura  
senal_periodica
```

desviacion\_periodica  
indice\_oscilatorio  
incidente  
tipo\_incidente  
estado  
cifrado  
direccion

## 20. Graficas generadas

Las graficas se guardan individualmente en pdi\_slp\_graficas.

```
| Archivo | Materia | Que muestra |  
|---|---|---|  
| 01_paquetes_vs_tiempo.png | Fundamentos matematicos | Paquetes por segundo. |  
| 02_intentos_login_vs_tiempo.png | Fundamentos matematicos | Intentos de login por segundo. |  
| 03_temperatura_vs_tiempo.png | Fisica | Temperatura con y sin enfriamiento. |  
| 04_estado_sistema_vs_tiempo.png | Fundamentos matematicos | Cambios de estado del sistema. |  
| 05_indice_flujo_digital_vs_tiempo.png | Algebra y trigonometria | Indice de flujo digital. |  
| 06_derivadas_trafico_vs_tiempo.png | Calculo | Primera y segunda derivada con sigmoide. |  
| 07_analisis_oscilatorio_vs_tiempo.png | Algebra y trigonometria | Onda regular contra comportamiento real. |
```

## 21. Grafica 1: paquetes

Entra:

segundo  
paquetes

Sale una linea de paquetes por segundo. Sirve para ver picos de trafico y compararlos con umbrales.

## 22. Grafica 2: intentos login

Entra:

segundo  
intentos\_login

Muestra intentos de acceso. Cuando sube demasiado puede indicar fuerza bruta.

## 23. Grafica 3: temperatura

Entra:

temperatura\_sin\_enfriamiento  
temperatura  
refrigeracion\_activa

Muestra que hubiera pasado sin enfriamiento y que paso despues de aplicar refrigeracion pasiva o activa.

## 24. Grafica 4: estado

Convierte estados a numeros:

NORMAL = 0  
SOSPECHOSO = 1  
ALERTA = 2  
CRITICO = 3

Sirve para ver en que segundos el sistema cambio de condicion.

## 25. Grafica 5: flujo digital

Muestra el resultado de:

$F = 0.45P + 0.25L + 0.15C + 0.15D$

Sirve para ver la intensidad general del evento.

## 26. Grafica 6: derivadas

Muestra:

primera derivada con sigmoide  $P'(t)$   
segunda derivada con sigmoide  $P''(t)$

La sigmoide hace que la grafica se enfoque mas en subidas fuertes y aceleraciones.

## 27. Grafica 7: analisis oscilatorio

Muestra dos cosas:

onda de envio regular  
comportamiento lineal real

La grafica se usa para interpretar el patron, y el indice oscilatorio calculado en el programa tambien puede apoyar el estado del sistema cuando la desviacion es muy alta.

## 28. CSV

m07\_reporte\_tabla.py genera:

reportes\_tabla/pdi\_slp\_reporte\_tabla.csv

Usa csv.DictWriter. Se escriben encabezados y despues una fila por cada ciclo.

## 29. Monitor

m05\_monitor.py imprime:

? pantalla inicial.  
? linea por cada ciclo.  
? panel completo cada 5 ciclos.  
? resumen final.

No toma decisiones. Solo muestra datos ya calculados.

## 30. Resumen del flujo

trafico normal  
-> posible ataque  
-> calculos matematicos  
-> estado y alertas  
-> protecciones  
-> historial  
-> graficas y CSV

El dato mas importante durante la simulacion es paquete.

El dato mas importante al final es estado\_actual["historial"].

## 31. Referencias para librerias

Estas referencias son para justificar el uso de herramientas, no para decir que la logica del sistema salio de ahi.

? Python random: <https://docs.python.org/3/library/random.html>

? Python time: <https://docs.python.org/3/library/time.html>

? Python math: <https://docs.python.org/3/library/math.html>

? Python csv: <https://docs.python.org/3/library/csv.html>

? Python os: <https://docs.python.org/3/library/os.html>

? Matplotlib: <https://matplotlib.org/stable/>